Yuri Dubovitsky

AI Engineering Harness for Healthcare Data Import

Design Walkthrough

Yuri Dubovitsky

Why this problem is hard

1. Healthcare imports are messy: duplicate patients, inconsistent date formats, missing identifiers, etc
2. Manual pipelines are brittle
3. AI-generated code needs **validation + iteration**

We need a system that can safely generate and refine code.

What I set out to build:

A reusable AI engineering harness that executes workflows without changing code.

- **Workflow-driven (not hardcoded)**
- **Supports iteration**
- **Designed for experimentation**

Key Idea:

The harness executes a sequence of steps defined externally.

No code changes needed to:

* change prompts
* change logic
* change validation strategy

3 step types only:

1. AI step
2. Build step
3. Run step

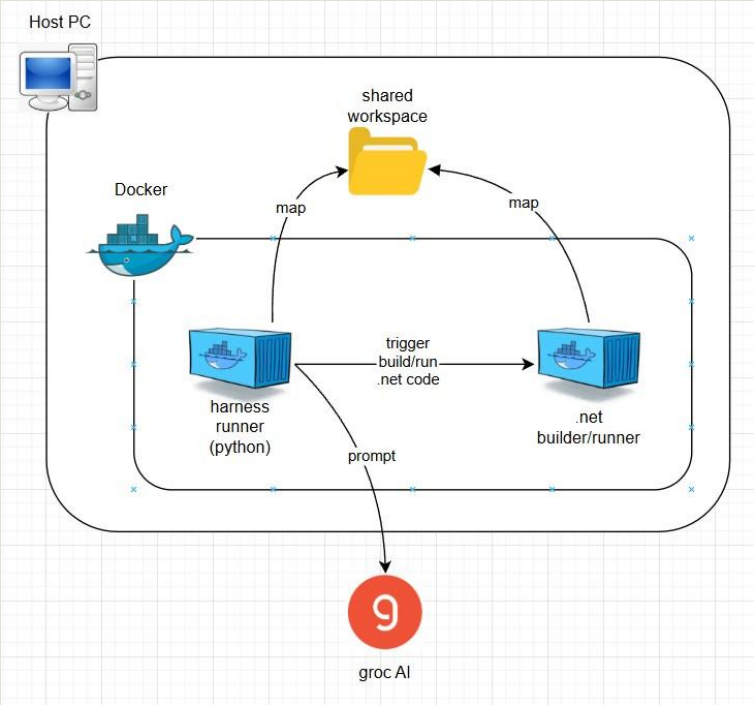
* This minimal set keeps the system simple, while still allowing extension with new step types when needed.

Yuri Dubovitsky

Architecture Overview

Everything communicates through artifacts - no tight coupling.

* this is not production grade architecture but PoC

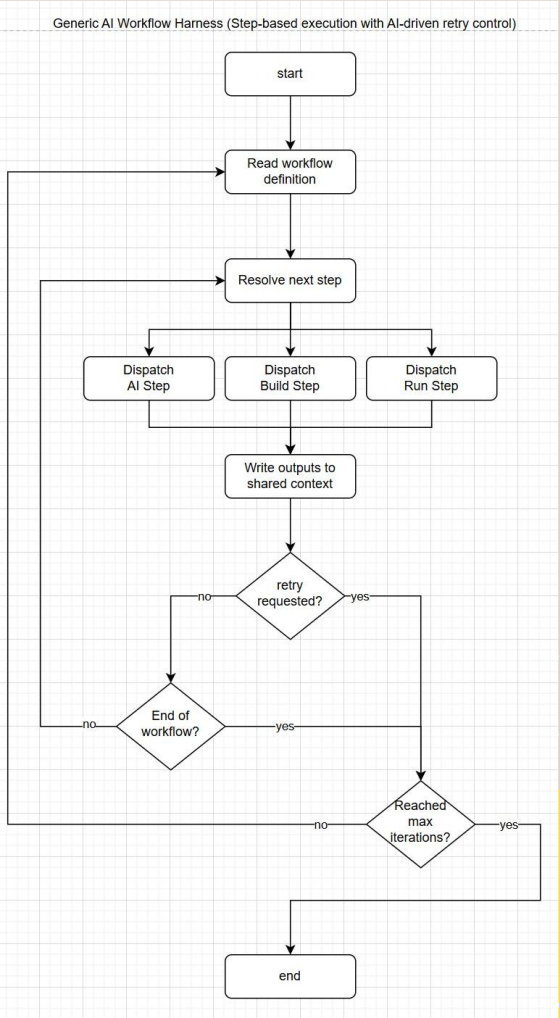


Yuri Dubovitsky

Workflow concept

How a workflow is defined

- JSON-based workflow
- Each step:
  - type (AI / build / run)
  - inputs
  - outputs
  - dependencies



Task Input as the Single Source of Truth

Separates stable task intent from dynamic prompt evolution.

The GitHub Issue is treated as the authoritative task definition driving the entire workflow.

- **Defines:**
  - requirements
  - edge cases
  - expected output structure
- **Can be consumed by multiple AI steps:**
  - code generation
  - error analysis
  - output validation
- **Persisted across iterations:**
  - remains unchanged while prompts evolve
  - reused in every cycle of the workflow
- **Used for validation:**
  - acts as ground truth for evaluating results
  - ensures consistency regardless of prompt modifications

Yuri Dubovitsky

Workflow Implementation: patient_import_v3

Step 1

Design Prompt (AI)
Generate an optimized coding prompt based on task definition and sample data.

Step 2

Generate Code (AI)
Produce the .NET pipeline implementation using the generated prompt.

Step 3,4

Build & Run (Build / Run)
Compile and execute the generated code against sample input data.

Step 4

Analyze & Improve (AI)
Evaluate build results and output, generate improved prompt, and trigger iteration if needed.

Goal

Validated Data Pipeline Output
A working import pipeline that handles edge cases and produces validated structured output.

JSON

```json
{
  "workflow_id": "patient-import-v3",
  "max_iterations": 3,

  "steps": [
    {
      "id": "design_prompt",
      "type": "ai",
      "model": "...",
      "inputs": { ... },
      "outputs": { ... }
    },
    {
      "id": "generate_code",
      "type": "ai",
      ...
    },
    {
      "id": "build_code",
      "type": "build"
    },
    {
      "id": "run_code",
      "type": "run"
    },
    {
      "id": "analyze_build",
      "type": "ai",
      "retry_on_failure": true
    }
  ]
}
```

Yuri Dubovitsky

Workflow Definition (JSON-driven execution)

Workflow Configuration

`workflow_id` → identifies workflow

`max_iterations` → controls retry loop

Step Definition

`type` → AI / build / run

`inputs` → files + artifacts

`outputs` → generated results

Iteration Control

`retry_on_failure`

improved prompt extracted from analysis

AI Steps

* use prompts + source files
* generate code or analysis

Execution Steps

* build → compile code
* run → execute pipeline

The entire execution logic is defined declaratively
No orchestration code changes required.

Demo

Yuri Dubovitsky

Next Iterations & Improvements

Workflow & Orchestration

Support parallel and conditional steps

Add richer dependency management

Enable dynamic workflow generation from task input

AI Strategy

Improve prompt versioning and tracking

Introduce multiple models per step

Add evaluation metrics for AI output quality

Validation & Safety

Schema-based validation (FHIR structure)

Deterministic validation alongside AI validation

Better handling of edge cases (deduplication, identifiers)

Developer Experience

UI for workflow design and monitoring

Better visibility into artifacts and iterations

Replay/debug individual steps

The goal is to evolve from a prototype harness into a reliable AI-driven engineering system.

Questions?